Deploying an Active Record Migration may take a lot of time depending on how fast your code review and software release process is, or on how many database changes the DDL statement needs to run for.

If you are very uncomfortable with the idea of direct database modifications, that's ok, your intuition is right. You want changes to be managed with a change management process, reviewed by team members. However, here you're looking to shortcut all of that and build your SQL DDL skills.

From psql, create an index called test_index for the tbl table you created.

This index should cover the col column on the tbl table and have the default B-Tree type. Don't worry about the purpose of the index; right now you're just getting comfortable developing and running the statements.

Type \timing to toggle on timing.

With timing enabled you'll see how long the CREATE INDEX statement runs. On large tables it can take a long time to create an index, so it's good to have an idea of how long this will take or sometimes test it in a lower environment.

Adding indexes on live systems in production should be done using the CONCURRENTLY keyword most of the time, which means that the index creation won't lock the table. However, using CONCURRENTLY makes the operation take about twice as long!

Try and write the SQL on your own first. When you're done or get stuck, refer to this SQL statement:

sql/create_index_basic.sql

```
CREATE INDEX test_index ON tbl (col);
```

Try dropping the index you just created using DROP INDEX. From psql, run DROP INDEX test_index;. Note that indexes can be dropped concurrently as well, which is a good idea on a live production system!

Rolling Back Schema Modifications

PostgreSQL has a feature called *Transactional DDL*.[11] Transactional DDL means that when structure changes are made, they either fully succeed or are fully rolled back depending on whether the transaction ends in a COMMIT or ROLLBACK. Transactional DDL is a key feature of PostgreSQL because it means that Active Record Migrations that change the database will succeed or fully roll back, and avoid a partially applied modification.

11. https://wiki.postgresql.org/wiki/Transactional_DDL_in_PostgreSQL:_A_Competitive_Analysis